

Debre Berhan University

Department of Software Engineering

Fundamentals of Software Security (SEng7431) Lecture Note

Lec02: Overview of Cryptography

Outline

- 1 Chapter2:Overview of Cryptography
 - Introduction to Computer security
 - Security Attacks,Services and Mechanisms
 - Classical cryptosystem
 - Historical Ciphers
 - crypt analysis Encryption Techniques
 - Symmetric and Public Key Cryptography
 - block cipher principles
- 2 Review of Shared Key Cryptography and Hash Functions
 - Review of Public key cryptosystem
 - Public Key Infrastructure(PKI)
 - Hash Function

Introduction to Computer security

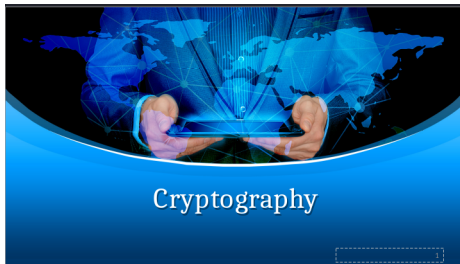
What is Computer Security?

- **Computer security** is security applied to computing devices such as **computers** and **smartphones**, as well as computer networks such as private and public networks.
- covers all the processes and mechanisms by which digital equipment, information and services are protected from **unintended or unauthorized access**, change or destruction.
- Traditionally, computer facilities have been physically protected for three reasons:
 - to prevent theft of or damage to the hardware
 - to prevent theft of or damage to the information
 - to prevent disruption of service

Chapter2: Overview of Cryptography

Review of Shared Key Cryptography and Hash Functions

- It includes physical security to prevent theft of equipment, and information security to protect the data on that equipment.
- It is sometimes referred to as "**IT security**", though these terms generally do not refer to physical security (locks and such).



- Computer data often **travels** from one computer to another, leaving the safety of its protected physical surroundings. Once the data is **out of hand**, people with bad **intention could modify** or forge your data, either for amusement or for their own benefit.
 - Cryptography can **reformat and transform** our data, making it **safer** on its trip between computers.
 - Technology is based on the essentials of **secret codes**, augmented by modern mathematics that protects our data in powerful ways..
- **Computer Security** - generic name for the collection of tools designed to protect data and to thwart hackers
- **Network Security** - measures to protect data during their transmission
- **Internet Security** - measures to protect data during their transmission over a collection of interconnected networks.

Security Attacks, Services and Mechanisms

- To assess the **security needs** of an organization effectively, the **manager** responsible for security needs some systematic way of defining the requirements for security and characterization of approaches to satisfy those requirements.
- One approach is to consider **three** aspects of information security:
 - **Security attack** – Any action that compromises the security of information owned by an organization.
 - **Security mechanism** – A mechanism that is designed to detect, prevent or recover from a security attack.
 - **Security service** – A service that enhances the security of the data processing systems and the information transfers of an organization. The services are intended to **counter security attacks** and they make use of one or more **security mechanisms** to provide the service.

Introduction to Classical cryptosystem

- **Cryptography** : is the strongest **tool** for controlling against many kinds of security threats or Well-disguised data cannot be read, modified, or fabricated easily.
- Consider the **steps involved** in sending messages from a **sender(S)** , to a **recipient(R)** .

Example

- If **S** deliver the message to **T**, who then delivers it to **R**, **T** then becomes the **transmission medium**.
- If an outsider(**O**), wants to **access the message** (to read, change, or even destroy it), we call **outsider(O)** an **interceptor** or **intruder**.
- Any time after **S** transmits it via **T**, the message is **vulnerable** to exploitation, and **O** might try to access the message in any of the following ways:

- **Block** : by preventing its **reaching** R, thereby affecting the **availability** of the message.
- **Intercept** : by **reading** or **listening** to the message, thereby affecting the **confidentiality** of the message.
- **Modify** : by seizing the message and **changing** it in some way, affecting the message's **integrity**.
- **Fabricate** : an **authentic-looking** message, arranging for it to be delivered as if it came **from S**, thereby also affecting the **integrity** of the message.
 - **Encryption** is a technique that can **address** all these problems.
 - We will study encryption as a **security technique**, and we see how it is used in **protecting** programs, databases, networks, and Electronic communications.

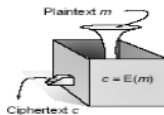
- **Cryptography** : is the art or science encompassing the **principles** and **methods** of transforming an **intelligible message** into one that is **unintelligible**, and retransforming that message to its original form
- Generally Cryptography is
 - art of **secret writing**.
 - art of **protection** using information.
 - science of **encrypting or hiding secrets**.
- Cryptography enables you to **store sensitive information** or **transmit it across insecure networks** (like the Internet) so that it **cannot be read** by anyone except the intended recipient.
 - a science of **encrypting** and **decryption** or hiding secrets.

- **Cryptanalysis:** The study of principles and methods of transforming an unintelligible message back into an intelligible message **without** knowledge of the **key**.
 - also called **code breaking**.
- An algorithm for transforming an intelligible into an unintelligible one using a **code-book**.

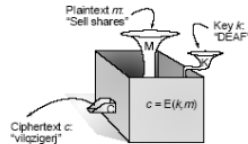
● Basic terminology of cryptography

- **Plaintext**:the original intelligible message
- **Cipher text**:the transformed message
- **Cipher**: an **algorithm** for transforming an intelligible message into one that is unintelligible by transposition and/or substitution methods
- **Key**: Some critical information used by the cipher, known only to the sender and receiver
- **Encipher(encode)**: The process of converting plaintext to cipher text using a **cipher** and a **key**
- **Decipher(decode)**: the process of converting cipher text back into plaintext using a **cipher** and a **key**

- **Example1: encryption**



- **Example2: decryption**



- **A. Cryptographic systems**

A system for encryption and decryption is called a **cryptosystem**. Cryptographic systems are generally **classified** along 3 independent dimensions:

1. **Type of operations used for transforming plain text to cipher text**

:

- All the encryption algorithms are based on two general principles:
 - A.Substitution**: in which each element in the plaintext is mapped into another element, and
 - B.Transposition**: in which elements in the plaintext are rearranged.

2. **The number of keys used** :If the sender and receiver uses same key then it is said to be **symmetric/single/conventional key**.

3. **The way in which plaintext is processed** :a block cipher process :ablock of element at a time and **stream cipher** one bit at a time.

1. Substitution cipher

- A substitution cipher **replaces** one symbol with another.
- **A key** determines what the substitution is E.g., 4, 8, 1, 26, ...
 - 1st letter in the alphabet will be represented by the 4th letter : **A-D**
 - 2nd letter will be represented by the 8th: **B-H**
 - 3rd letter will be represented by the 1st: **C-A**
 - 4th letter will be represented by the 26th : **D-Z**
- A substitution cipher replaces one symbol with another.
- Substitution ciphers can be categorized as either **monoalphabetic ciphers** or **polyalphabetic ciphers**.

A. Monoalphabetic Ciphers

- In monoalphabetic substitution, the **relationship** between a symbol in the plaintext to a symbol in the ciphertext is always **one-to-one**.

Example

1. Given a plaintext and find its corresponding ciphertext. know the cipher is probably **monoalphabetic** because both l (els) are encrypted as O.

Plaintext:hello ——— **Ciphertext: KHOOR**

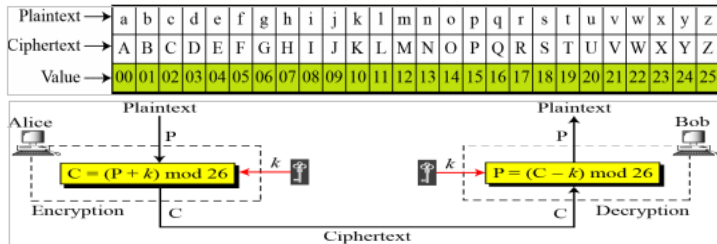
2. The following shows a plaintext and its corresponding ciphertext. The cipher is **not monoalphabetic** because each l (el) is encrypted by a different character.

Plaintext:hello ——— **Ciphertext: ABNZF**

- the different types of Monoalphabetic Ciphers: **Additive Cipher**

I. Additive Cipher

- The **simplest** monoalphabetic cipher is the additive cipher.
 - this cipher is sometimes called a **shift cipher** and sometimes, a Caesar cipher, but the term additive cipher better reveals its mathematical nature.
- Operation principle:** each letter is translated into the letter a **fixed number** of positions after it in the alphabet table.



- More formally it can be represented:
 - $\text{encrypt}(\text{Letter}, \text{Key}) = (\text{Letter} + \text{Key}) \pmod{26}$
 - $\text{decrypt}(\text{Letter}, \text{Key}) = (\text{Letter} - \text{Key}) \pmod{26}$
- **Note:** When the cipher is **additive**, the plaintext, ciphertext, and key are integers in Z_{26} . i.e. the key k is a number in the range **0-25**.

Example:1

Encrypt(**NIKITA**,3) = **QLNLWD**

Decrypt(**QLNLWD**,3) = **NIKITA**

- Generally **Shift Cipher and Caesar Cipher**
 - Historically, additive ciphers are called **shift ciphers**.
Example3: Julius Caesar **used** an additive cipher to communicate with his **officers**. For this reason, additive ciphers are sometimes referred to as the **Caesar cipher**. Caesar used a **key of 3** for his communications.

- **Question: is Caesar cipher secure?**

Of course NOT, since there are only 26 possible keys, it is easy to be broken by exhaustive key search.

NB:-The problem with the Caesar cipher is that the number of keys is too small.

- **Example2:-** Eve has intercepted the ciphertext “UVACLYFZLJBYL”. Show how she can use a brute-force attack to break the cipher.

- Eve tries keys from 1 to 7. With a key of 7, the plaintext is “not very secure”, which makes sense.

Ciphertext: UVACLYFZLJBYL

K = 1	→	Plaintext: tuzbkxeykiaxk
K = 2	→	Plaintext: styajwdxjhwj
K = 3	→	Plaintext: rsxzivcwigyvi
K = 4	→	Plaintext: qrwylubvfhxuh
K = 5	→	Plaintext: pqvxgtaugewtg
K = 6	→	Plaintext: opuwfsztfdsf
K = 7	→	Plaintext: notverysecure

- **Note:-** On average, a plaintext will be computed after trying $26/2=13$ times. it is possible to break the Caesar cipher by trial and error

II. Multiplicative Ciphers

- In a multiplicative cipher, the plaintext ,ciphertext and key are integers.
- Operation principles



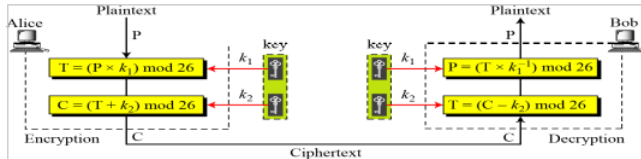
Example:-

We use a multiplicative cipher to encrypt the message **hello** with a key of 7. the ciphertext is **"XCZZU"**.

Plaintext: h $\rightarrow$ 07	Encryption: $(07 \times 07) \bmod 26$	ciphertext: 23 $\rightarrow$ X
Plaintext: e $\rightarrow$ 04	Encryption: $(04 \times 07) \bmod 26$	ciphertext: 02 $\rightarrow$ C
Plaintext: l $\rightarrow$ 11	Encryption: $(11 \times 07) \bmod 26$	ciphertext: 25 $\rightarrow$ Z
Plaintext: l $\rightarrow$ 11	Encryption: $(11 \times 07) \bmod 26$	ciphertext: 25 $\rightarrow$ Z
Plaintext: o $\rightarrow$ 14	Encryption: $(14 \times 07) \bmod 26$	ciphertext: 20 $\rightarrow$ U

III. Affine Ciphers

- Operational principle



$$C = (P \times k_1 + k_2) \bmod 26$$

$$P = ((C - k_2) \times k_1^{-1}) \bmod 26$$

where k_1^{-1} is the multiplicative inverse of k_1 and $-k_2$ is the additive inverse of k_2

- Example1:** Use an affine cipher to encrypt the message “hello” with the key pair(7,2).

P: h → 07	Encryption: $(07 \times 7 + 2) \bmod 26$	C: 25 → Z
P: e → 04	Encryption: $(04 \times 7 + 2) \bmod 26$	C: 04 → E
P: l → 11	Encryption: $(11 \times 7 + 2) \bmod 26$	C: 01 → B
P: l → 11	Encryption: $(11 \times 7 + 2) \bmod 26$	C: 01 → B
P: o → 14	Encryption: $(14 \times 7 + 2) \bmod 26$	C: 22 → W

- **Example2** Use the affine cipher to decrypt the message “ZEBBW” with the key pair(7, 2) in modulus 26.

C: Z → 25	Decryption: $((25 - 2) \times 7^{-1}) \bmod 26$	P: 07 → h
C: E → 04	Decryption: $((04 - 2) \times 7^{-1}) \bmod 26$	P: 04 → e
C: B → 01	Decryption: $((01 - 2) \times 7^{-1}) \bmod 26$	P: 11 → l
C: B → 01	Decryption: $((01 - 2) \times 7^{-1}) \bmod 26$	P: 11 → l
C: W → 22	Decryption: $((22 - 2) \times 7^{-1}) \bmod 26$	P: 14 → o

- **Note:**
 - additive, multiplicative, and affine ciphers have small key domains, due to this they are very **vulnerable to brute-force attack**. A better solution is use **polyalphabetic substitution**.

B. Polyalphabetic Ciphers

- In polyalphabetic substitution, each **occurrence** of a character may have **a different substitute**.
 - relationship between a character in the plaintext to a character in the ciphertext is **one-to-many**.
 - make each ciphertext character **dependent** on both the corresponding plaintext character and its position in the plaintext. Thus, our key should be **a stream of subkeys**.

I. Autokey Cipher

- One of the type of polyalphabetic substitution is autokey cipher
- Operational principle**

$$P = P_1 P_2 P_3 \dots, C = C_1 C_2 C_3 \dots, K = (K_1, P_1, P_2) \dots$$

$$\text{Encryption} : C_i = (P_i + K_i) \bmod 26 \text{ and}$$

$$\text{Decryption} : P_i = (C_i - K_i) \bmod 26$$

- Example1**

Assume that Alice and Bob agreed to use an autokey cipher with initial key value **k1 = 12**. Now Alice wants to send Bob the message "**Attack is today**". Enciphering is done character by character.

Plaintext:	a	t	t	a	c	k	i	s	t	o	d	a	y
P's Values:	00	19	19	00	02	10	08	18	19	14	03	00	24
Key stream:	12	00	19	19	00	02	10	08	18	19	14	03	00
C's Values:	12	19	12	19	02	12	18	00	11	7	17	03	24
Ciphertext:	M	T	M	T	C	M	S	A	L	H	R	D	Y

II.Vigenere Cipher

- Operational principle

- This scheme of cipher uses a text string (say, a word) as a **key**,
- The secret key is a short sequence of letters (**e.g. a word**).
 - Encryption involves **adding** the **plaintext** letter to a **key letter**, with the key letters used in rotation.

$P = P_1P_2P_3 \dots$	$C = C_1C_2C_3 \dots$	$K = [(k_1, k_2, \dots, k_m), (k_1, k_2, \dots, k_m), \dots]$
Encryption: $C_i = P_i + k_i$		Decryption: $P_i = C_i - k_i$

- Note:-that the Vigenere key stream does **not depend** on the plaintext characters.
 - it depends only on the **position** of the character in the plaintext.

● Process of Vigenere Cipher

- The sender and the receiver decide on a key.
- **Example1:** Let **point** is the key. Numeric representation of this key is **16 15 9 14 20**.
- The sender wants to encrypt the message, **attack from south east**. He will arrange plaintext and numeric key as follows.

a	t	t	a	c	k	f	r	o	m	s	o	u	t	h	e	a	s	t
16	15	9	14	20	16	15	9	14	20	16	15	9	14	20	16	15	9	14

- now shifts each plaintext alphabet by the number written below it to create ciphertext as shown below

a	t	t	a	c	k	f	r	o	m	s	o	u	t	h	e	a	s	t
16	15	9	14	20	16	15	9	14	20	16	15	9	14	20	16	15	9	14
Q	I	C	O	W	A	U	A	C	G	I	D	D	H	B	U	P	B	H

- **Example2**:-if the key is **SESAME**, encryption works as follows:

T H I S I S A T E S T M E S S A G E

Message

S E S A M E S E S A M E S E S A M E

Keystream

L L A S U W S X W S F Q W W K A S I

Ciphertext

1

- NB:- '**A**' will encrypt to a **different letter** depending on where it is in the message.

III. Vernam Cipher(One-Time Pad)

- Gilbert Vernam patented this cipher in 1917 for encryption and decryption of **telegraph messages**.
 - To **send** a binary string **M** you need a key **k** as long as the message.
 - Each key **k** is **uniformly** selected at **random** and can be used only once hence one-time pad.
 - **Encryption: $c = m \text{ XOR } k$.**
 - **Decryption: $m = c \text{ XOR } k$.** i.e. exclusive-or (XOR):

XOR	0	1
0	0	1
1	1	0

- There are **two special cases** of Vigenere cipher.
 - The **keyword length** is same as **Plaintext message**. This case is called **Vernam Cipher**.
 - a **keyword** is a randomly generated string of alphabets and is used **only once**.
 - It is more **secure** than typical Vigenere cipher.
- **Example1**: Let us say, we encrypt the name **“point”** with a one-time pad. It is a **5 letter** text.
 - to **break** the cipher text by **brute force**, you need to try all possibilities of keys and conduct computation for $(26 \times 26 \times 26 \times 26 \times 26) = 26^5 = 11881376$ times. for a message with 5 alphabets.
 - a **longer message**, the computation grows exponentially with every additional alphabet. makes it impossible **to break** the cipher text by brute force.

- **Example1:** let the encrypted message **WORLD** with key **POINT**

plaintext	W	O	R	L	D
	22	14	17	11	03
Binary bit :	00010100	00001110	00001111	00001011	00000011
Key	P	O	I	N	T
	15	14	08	13	19
binary bit	00001111	00001110	00001000	00001101	00010011
XOR	00011011	00000000	00000111	00000110	00010000
	27	00	07	06	16
Ciphertext	B	A	H	G	Q

- First encryption: $c1 = m1 \text{ XOR } k.$
- Second encryption: $c2 = m2 \text{ XOR } k.$
- $c1 \text{ XOR } c2 = m1 \text{ XOR } m2$
- Vulnerable to frequency analysis. ²

² Example1: Vernam Cipher (One-Time Pad)

2. Transposition Ciphers

- does not substitute one symbol for another; instead it changes the location of the symbols.
- hide the message by rearranging the letter order.
- a transposition cipher records symbols.

2.1 Keyless Transposition Ciphers

- Simple transposition ciphers, which were used in the past, are **keyless**.
- **Operation principle**
 - 1 text is written into a table column by column and then is transmitted row by row.
 - 2 text is written into a table row by row and then is transmitted column by column.

- **Example1:**

Plaintext = "HELLO WORLD"

| H W

E O

L R

L L

O D

Ciphertext = "HWEOLRLLOD"

- **Example2:**

Alice and Bob can agree on the number of columns and use the second method. Alice writes the same plaintext, row by row, in a table of four columns.

m	e	e	t
m	e	a	t
t	h	e	p
a	r	k	

- She then creates the ciphertext "MMTAEEHREAETTP".

● Example3

The cipher in the above **Example2** is actually a transposition cipher. The following shows the permutation of each character in the plaintext into the ciphertext based on the positions.

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
01	05	09	13	02	06	10	13	03	07	11	15	04	08	12

- The second character in the plaintext has moved to the fifth position in the ciphertext; the third character has moved to the ninth position; etc.
- Although the characters are **permuted**, there is a pattern in the permutation:

(01, 05, 09, 13), (02, 06, 10, 14), (03, 07, 11, 15), and (04, 08, 12).

2.2 Keyed Transposition Ciphers

- The keyless ciphers permute the characters by using writing plaintext in one way and reading it in another way.
- The permutation is done on the whole plaintext to create the whole ciphertext.
- Another method is to divide the plaintext into groups of predetermined size, called **blocks**, and then use **a key to permute** the characters in each block separately.

- **Example 1:** - Alice needs to send the message **Enemy attacks tonight** to Bob.

e n e m y a t t a c k s t o n i g h t z

- key used for encryption and decryption is **a permutation key**, which shows how the characters are permuted.

- The key used for encryption and decryption is a permutation key, which shows how the characters are permuted.

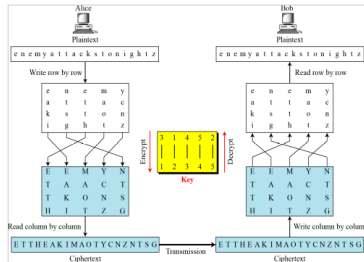
Encryption	↓	<table><tr><td>3</td><td>1</td><td>4</td><td>5</td><td>2</td></tr><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	3	1	4	5	2	1	2	3	4	5	↑	Decryption
3	1	4	5	2										
1	2	3	4	5										

- The permutation yields

E E M Y N T A A C T T K O N S H I T Z G

2.3 Combining Two Approaches

● Example1



- a single key was used in two directions for the column exchange: downward for encryption, upward for decryption. It is customary to create two keys.

Introduction to Cryptanalysis

What is Crypt analysis ?

- is process of breaking encryption without **knowing key**.
- **Cryptographic attacks** are designed to **subvert** the **security of cryptographic algorithms**, and they are used to attempt to decrypt data without **prior access to a key**.
 - It is an art of **deciphering encrypted data**.
 - **cryptanalysis** and **cryptography** (the art of creating **hidden writing**, or ciphers) form the science of **Cryptology**.
 - **General approaches**.
 - ① Frequency Analysis attack
 - ② Brute Force attack
 - ③ Cryptanalytic attack

1. Frequency Analysis attack

- Guess values based on frequency of occurrence.

- **Example1:**

Ciphertext:-HSPAA SLRUV DSLKN LPZHK HUNLY VBZAO PUN

Find most frequent letters

- 4 times: L
- 3 times: A, H, N, P, S, U

- **Guess:**

Decrypt(L) = E

- **Key** = L-E = 7

then

- **Decrypt(HSPAA SLRUV DSLKN LPZHK HUNLY VBZAO PUN, 7) =**
ALITT LEKNO WLEDG EISAD ANGER OUSTH ING

2. Brute force attack : is systematically attempts every possible key

- it is most often used in a known plaintext or ciphertext-only attack.
- always possible to **simply try every key**.
- Given a **finite** key length and sufficient time, a brute force attack is always successful.

Key Size (bits)	Number of Alternative Keys	Time required at 1 decryption/ μ s	Time required at 10^6 decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

- Here is an example of a brute force attack on a given ciphertext:
 - Example1** given that Ciphertext = **IGKYGXOYOTYKIAXK**
 - Decrypt(IGKYGXOYOTYKIAXK, 1) = HFJXFWNXNSXJHZWJ
 - Decrypt(IGKYGXOYOTYKIAXK, 2) = GEIWEVMWMRWIGYVI
 - Decrypt(IGKYGXOYOTYKIAXK, 3) = FDHVDULVLQVHFXUH
 - Decrypt(IGKYGXOYOTYKIAXK, 4) = ECGUCTKUKPUGEWTG
 - Decrypt(IGKYGXOYOTYKIAXK, 5) = DBFTBSJTJOTFDVSF
 - Decrypt(IGKYGXOYOTYKIAXK, 6) = CAESARISINSECURE
- Here is an example of a brute force attack on a 4-bit key:

0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	1	0	0	1	1	1	1
1	0	0	0	0	1	0	0	1	1	0	1	0	1	1	0	0	1	1	0	1	1	1	0	1	1	1	0	1	1	1	1

3. Cryptanalytic attack

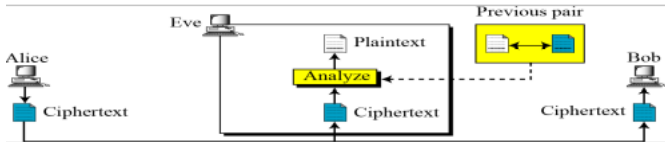
- There are six related cryptographic attack methods, including three plaintext-based methods and three ciphertext-based methods.

Plaintext-Based Attacks	Known Plaintext	Chosen Plaintext	Adaptive Chosen Plaintext
Ciphertext-Based Attacks	Ciphertext Only	Chosen Ciphertext	Adaptive Chosen Ciphertext

- The different types of cryptanalytic attack that is based on the plaintext and ciphertext attacks
 - known Plaintext
 - ciphertext Only
 - chosen Plaintext
 - chosen Ciphertext
 - Adaptive Chosen Plaintext and Adaptive Chosen Ciphertext Attacks

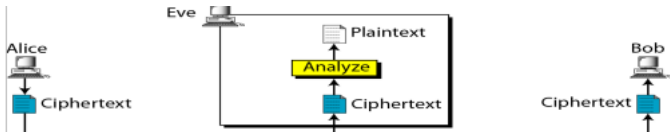
A. Known Plaintext

- A known plaintext attack: is an attack where a cryptanalyst has access to a plaintext and the corresponding ciphertext and seeks to discover a correlation between the two.



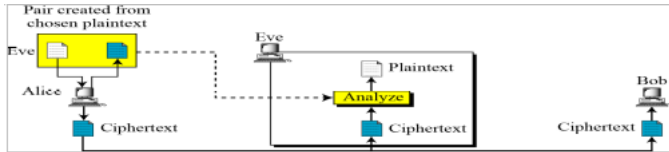
B. Ciphertext Only

- a ciphertext-only attack: is an attack where a cryptanalyst has **access** to a ciphertext but **does not** have access to corresponding plaintext.
- With simple ciphers, such as the Caesar Cipher, frequency analysis can be used to **break** the cipher.



C. Chosen Plaintext

- A chosen plaintext attack: is an attack where a cryptanalyst can **encrypt a plaintext** of his choosing and study the resulting ciphertext.
- This is most common **against** asymmetric cryptography, where a cryptanalyst has access to a **public key**.



E. Adaptive chosen plaintext and adaptive chosen ciphertext Attacks

- In both adaptive attacks, a cryptanalyst chooses further plaintexts or ciphertexts (adapts the attack) based on prior results.

Type of Attack	Known to Cryptanalyst
Ciphertext Only	• Encryption algorithm • Ciphertext
Known Plaintext	• Encryption algorithm • Ciphertext • One or more plaintext-ciphertext pairs formed with the secret key
Chosen Plaintext	• Encryption algorithm • Ciphertext • Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen Ciphertext	• Encryption algorithm • Ciphertext • Ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

Symmetric and Public Key Cryptography

- Many **encryption algorithms** are widely available and used in information security.
- can be categorized into **two Symmetric(private) and Asymmetric(public) keys encryption.**
 - **Symmetric keys/secret key** encryption: **only one key** is used to encrypt and decrypt data.
 - **Asymmetric keys**, **two keys** are used: **private and public keys.**
 - Public key is used for **encryption** and private key is used for **decryptions**(RSA and Digital Signatures).
 - Because **users tend** to use two keys: public key, which is known to the public and private key which is known only to the **user**

A.Symmetric-Key Cryptosystems(SKC)

● Symmetric-Key Cryptosystems(SKC)

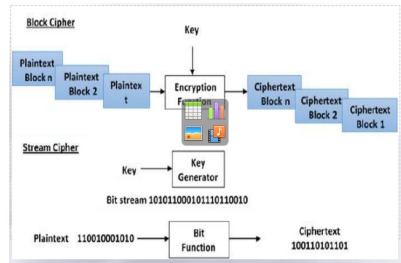
- Uses conventional / secret-key / single-key.
- The secret key **shared** by both sender and receiver.
- All classical encryption algorithms are **private-key**.
- Use the **same** key(shared key) for encryption and decryption process.
- **Basic features of symmetric key**
 - use the same key to encrypt and decrypt the data.
 - Key length are **short** (40 bits- 256 bits)
 - Faster than Asymmetric encryption
 - Commonly used for encryption **bulk data** such as in VPN traffic

Symmetric encryption Algorithm

- **Encryption Algorithm** often classified in to
 - **Block Ciphers**
 - In this scheme, the **plain binary text** is processed in **blocks groups** of bits at a time; i.e. a block of plaintext bits is selected, a series of operations is performed on this block to generate a block of cipher text bits.
 - The number of bits in a block is fixed.
 - **Example:** the schemes **DES** and **AES** have block sizes of 64 and 128, respectively.
 - **Stream Ciphers** :In this scheme, the plaintext is processed **one bit** at a time i.e. one bit of plaintext is taken, and a series of operations is performed on it to generate one bit of cipher text.

Example1: Let see the two types of symmetric encryption schemes

- There is a vast number of block ciphers schemes that are in use.
- Many of them are publically known.
 - Digital Encryption Standard
 - Triple DES
 - Advanced Encryption Standard AES



Feistel Block Cipher

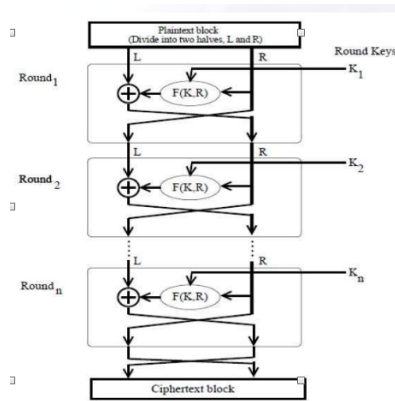
- Feistel Cipher is not a **specific scheme** of block cipher.
- It is a **design model** from which many different block ciphers are **derived**.
Example: DES is just one example of a Feistel Cipher.
- A cryptographic system based on Feistel cipher structure **uses the same key** for both encryption and decryption.
- **Encryption Process**
 - The encryption process uses the Feistel structure **consisting multiple rounds** of processing of the plaintext.
 - each round consisting of a “**substitution**” followed by a permutation step.
 - **input block** to each round is divided into **two halves** i.e **left half(L)** and the **right half(R)**.

- In each round

- The right half of the block(R), goes through **unchanged** , But
- The left half(L), goes through an operation that **depends** on R and the encryption key.
 - first, we apply an **encrypting function(f)** that takes two input i.e the **key(K)** and R , then
 - the function produces the output **$f(R, K)$** and XOR the output of the mathematical function with L .
- In **real implementation** of the Feistel Cipher, such as DES, instead of using the **whole encryption key** during each round, a **round-dependent key** a sub key is **derived** from the encryption key.
 - This means that **each round** uses a different key, although all these **sub keys** are related to the **original key**.

- The **permutation step** at the end of each round **swaps** the modified L and unmodified R ,therefore,
 - the **L** for the next round would be **R** of the current round.
 - R for the next round be the output L of the current round.
- The above substitution and permutation **steps** form **a round**.
 - The number of rounds are **specified** by the **algorithm design** .
- Once the **last round** is completed then the **two sub blocks(R and L)**,are **concatenated** in this order to form the ciphertext block.

- Look at the following diagram



● Decryption Process

- The process of decryption in **Feistel cipher** is almost **similar** to the encryption process.
- Instead of starting with a **block of plaintext**, the **ciphertext block** is **fed** to start the process.
- In the **case of decryption**, the only difference is that the **subkeys** used in encryption are used in the **reverse order** .
 - The key used in **last round** in encryption, it used in **first round** in decryption.
 - The final **swapping** of **L** and **R** in last step of the Feistel Cipher is essential.
 - If these are **not** swapped then the resulting ciphertext could not be decrypted using the same algorithm

● Number of Rounds

- The **number of rounds** used in a Feistel Cipher **depends** on desired security from the system.
- More number of rounds provide more **secure** system.
- But at the same time, more rounds mean the **inefficient slow** encryption and decryption processes.
- Number of rounds in the systems thus **depend** upon **efficiency–security tradeoff** .

A. Data Encryption Standard (DES)

- DES algorithm purpose is to **provide** a standard method for protecting sensitive commercial and unclassified data.
- The most common **SKC scheme** used today, DES was designed by IBM in the 1970s.
- **DES algorithm consists of the following basic points**
 - DES **Utilizes** block cipher, which means that during the encryption process, the plaintext is **broken** into fixed length blocks of 64 bits.
 - The key is **56 bits** wide. **8-bit out** of the total 64-bit block key is used for **parity check** (for example, each byte has an odd number of bits set to 1).
 - involves **carrying** out combinations, **substitutions** and **permutations** between the **text to be encrypted** and the **key**.

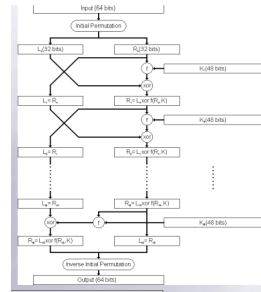
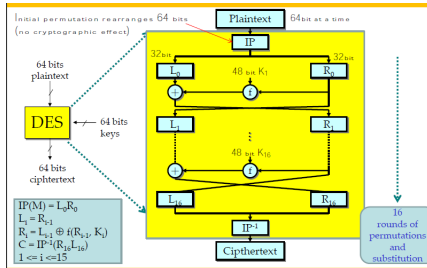
- DES is the **most widely** used symmetric algorithm despite claims whether **56 bits** is **long** enough to guarantee security.
- Using current technology, **56-bit key** size is vulnerable to a **brute force attack** .
- **DES Encryption process**
 - starts with an **initial permutation (IP)** of the 64 input bits.
 - These bits are then divided into **two 32-bit** halves called **L** and **R**.
 - The encryption then proceeds through **16 rounds**, each using the L and R parts, and a **subkey**.
 - The **R** and **subkeys** are processed in the so called **f-function** , and exclusive or(XoR) of the output of the **f-function** with the **existing L** part to create the **new R** part.
 - The new L part is simply a copy of the **incoming R part**.

- In the **final round**, **L** and **R** parts are **swapped** once more before the **final permutation (FP)** producing the output block.
 - Decryption is **identical** to encryption, except that the **subkeys** are used in the **opposite order** .
 - That is, **subkey 16** is used in round 1, subkey 15 is used in round 2, etc., ending with subkey 1 being used in round 16.
- As with any **encryption scheme**, there are **two inputs** to the encryption function: the **plaintext** to be encrypted and the **key**.

Chapter2: Overview of Cryptography

Review of Shared Key Cryptography and Hash Functions

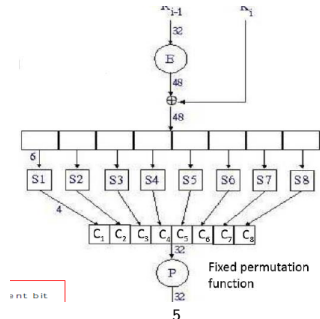
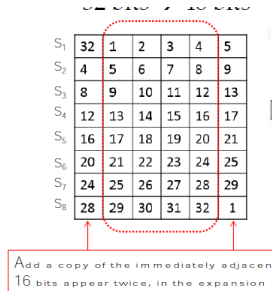
- The **plaintext** must be 64 bits in length and the key is **56 bits** in length.



4

- The **f-function** **mixes** the bits of the **R portion** using the **Subkey** for the current round.
- First the **32-bit R value** is expanded to **48 bits** using a **permutation E(Expanded function)**. That value is then **exclusive-or'ed** with the subkey.
 - The **48 bits** are then divided into **eight 6-bit chunks**, each of which is fed into an **S-Box** that mixes the bits and produces a **4-bit output**. A little bit funny operation!!
 - Those 4-bit outputs are combined into a 32-bit value, and permuted once again to produce the **f-function output**.

- the 32-bit R value is expanded to 48 bits using a permutation E(Expanded function)



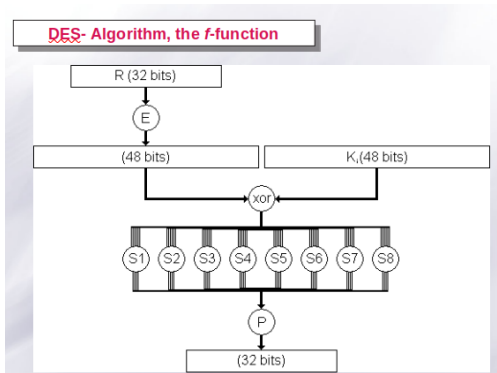
● Example

The S-Box

		S ₁															
		Column Number															
Row	No.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7	
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8	
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0	
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13	

If S_1 is the function defined in this table and B is a block of 6 bits, then $S_1(B)$ is determined as follows: The first and last bits of B represent in base 2 a number in the decimal range 0 to 3 (or binary 00 to 11). Let that number be i . The middle 4 bits of B represent in base 2 a number in the decimal range 0 to 15 (binary 0000 to 1111). Let that number be j . Look up in the table the number in the i -th row and j -th column. It is a number in the range 0 to 15 and is uniquely represented by a 4 bit block. That block is the output $S_1(B)$ of S_1 for the input B . For example, for input block $B = 011011$ the first bit is "0" and the last bit "1" giving 01 as the row. This is row 1. The middle four bits are "1101". This is the binary equivalent of decimal 13, so the column is column number 13. In row 1, column 13 appears 5. This determines the output; 5 is binary 0101, so that the output is 0101. Hence $S_1(011011) = 0101$.

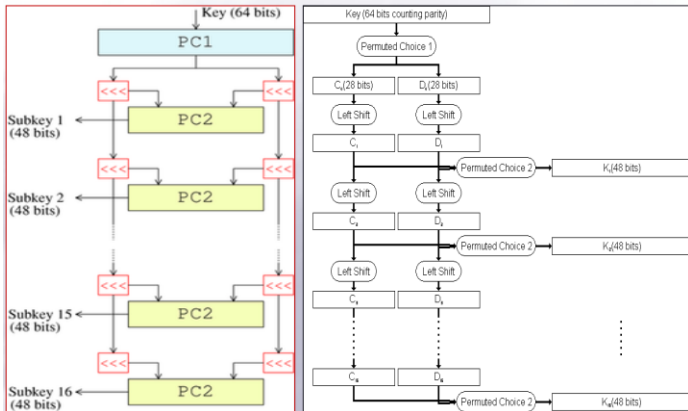
- DES Algorithm, the f-function



- **DES- Generating Subkey**

- To generate the **subkeys**, start with the **56-bit key** (64 bits if you include the parity bits).
 - These are permuted and divided into two halves called **C** and **D**.
 - For each round, C and D are each shifted left circularly one or two bits.
- The 48-bit subkey is then selected from the current C and D bits.

DES- Algorithm - Key Schedule and Subkey Generation :



Initial permutation and Final permutation

● Initial permutation

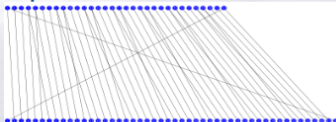
- The above table specifies the input permutation on a 64-bit block.
- The meaning is as follows:
 - the **first bit** of the **output** is taken from the **58th bit of the input**
 - the **second bit** from the **50th bit of the input** , and so on, with the **last bit of the output** taken from the **7th bit of the input**.
 - it is a vector, not a matrix.

● final permutation(FP) : is the **inverse** of the **IP** and interpreted similarly.

- the **output** of the FP has **bit 40** of the **preoutput block**
- as its first bit, **bit 8** as its second bit, and so on, until bit 25 of the **preoutput block** is the **last bit of the output**.

- DES- Permutation principles :

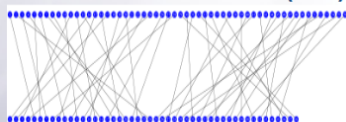
Expansion/Permutation



The 32-bit half-block of data is expanded to 48 bits.

E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

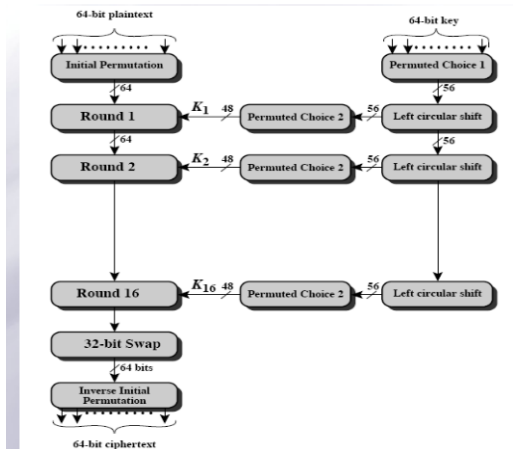
Contraction/Permuted Choice (PC-2)



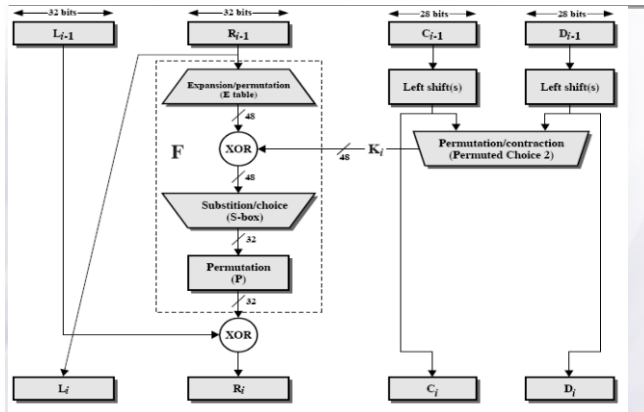
Selects/Extracts the 48-bit subkey for each round from the 56-bit key-schedule state.

PC-2					
14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

● **DES- Algorithm, General depiction :**



● **DES- Single round of DES Algorithm :**

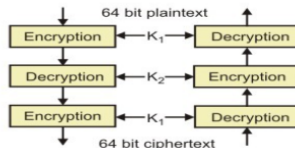


Generally DES

- Cracking: The most basic method of attack for any cypher is **brute force** - trying every possible key in turn.
- length of the key **determines** the number of possible keys, and hence the feasibility of the approach.
- DES is not adequate with this regard due to its key size
- In academia, various proposals for a DES-cracking machine were advanced. However, none of these early proposals were ever implemented.

Triple DES

- **Triple DES**, popularly known as **3DES** is used to make **DES** more secure By effectively increasing the **key length**. Its operation is explained below.
 - Each **block of plaintext** is subjected to encryption by **K1**, decryption by **K2** and again encryption by **K1** in a sequence as shown in below figure.
 - $C = E_{K1} [D_{K2} [E_{K1} [P]]]$ for encryption
 - $P = D_{K1} [E_{K2} [D_{K1} [C]]]$ for decryption



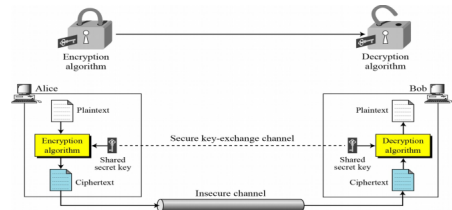
B. Advanced Encryption Standard(AES)

- **AES**: is a symmetric block cipher that is intended to **replace** DES as the approved standard for a wide range of applications.
- **AES Requirements**:
 - private key **symmetric block** cipher
 - **128** bit data, 128/192/256-bit keys
 - stronger and faster than **Triple DES**
 - has **9/11/13 rounds**.

Symmetric Cipher Model

- **Example1:** Figure below shows symmetric-key cryptography.

Alice can **send** a message to **Bob** over an insecure channel with the assumption that an adversary, **Eve cannot** understand the contents of the message.



6

- **Requirements symmetric encryption:** two requirements for secure use of symmetric encryption:
 - Ⓐ. Strong encryption algorithm
 - Ⓑ. Secret key known only to sender/receiver I.e
 $Y = E_K(X)$ and $X = D_K(Y)$
 - **Assume** encryption algorithm is **known**
 - Implies a secure channel to distribute key
 - **Problem:- Key distribution problem.**

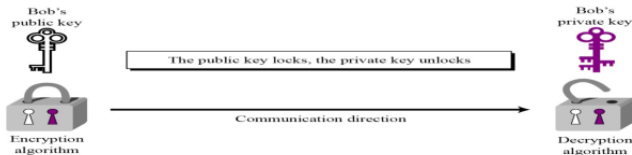
Key Distribution

- symmetric schemes **require** both parties to **share** a common secret key?
- issue is **how to** securely distribute this key.
- **Let Given two parties A and B have various key distribution alternatives:**
 - can **select** key and physically **deliver** to **B**.
 - **third party** can **select** and **deliver** key to **A** and **B**.
 - if **A** and **B** have **communicated** previously can **use** previous key to **encrypt** a new key.
 - if **A** and **B** have **secure** communications with a **third party C**, **C** can **relay** key between A and B.
- **Key Hierarchy:**
 - **session key:** temporary key, used for encryption of data between users and for **one** logical session then discarded.
 - **master key:** used to encrypt **session keys**, shared by user and key distribution center

B.Asymmetric-key Cryptosystem

- In **public key** cryptography, there are **two** keys: a private key and a public key.
 - The public key is **announced** to the public; whereas the private key is **kept** by the receiver.
 - sender **uses** the public key of the receiver for encryption and the receiver uses his **private key** for decryption.
 - It is also known as public-key encryption.
 - can be used for **confidentiality, authentication**, or both.
- **Public Key cryptosystem issues**
 - Plain text
 - Encryption algorithm
 - Public and private key
 - Cipher text

- Figure below shows the **general** idea of asymmetric-key cryptography as **used for confidentiality**.
- unlike symmetric-key cryptography, there are distinctive keys in asymmetric-key cryptography: a **private** key and a **public** key.
 - If encryption and decryption are thought of as **locking** and **unlocking** padlocks with keys, then the padlock that is locked with a public key can be unlocked only with the corresponding private key.
 - **Eve** should **not** be able to **advertise** her public key to the community pretending that it is Bob's public key.



- The essential steps are the following.



- Each user generates a pair of keys to be used for the encryption and decryption of messages.
- Each user places one of the two keys in a public register or other accessible file.
- If Bob wishes to send a **confidential message** to Alice, Bob encrypts the message using Alice's public key.

- When **Alice receives** the message, she decrypts it using her private key.
- No other recipient can decrypt the message because only Alice knows Alice's private key.

Example :

- Which one of the encryption or decryption key is made public depends on the use of the key
 - If Hana wants to send a **confidential message** to Ahmed
 - She encrypts the message using Ahmed's public key Send the message
 - Ahmed will then decode it using his own private key
 - On the other hand, if Ahmed needs to make sure that a **message sent** by Hana really comes from her, how can he make that?

● Using digital signature

- Hana has to first encrypt a digital signature using her private key
- Then encrypt the message (signature included) with Ahmed's public key
- Sends the encrypted message to Ahmed
- Ahmed decrypts the message using his private key
- Ahmed then decrypts the signature using Hana's public key If successful, he insures that it comes from Hana

Examples of Public-key Cryptosystems

- Diffie -Hellman key exchange
- Rivest , Shamir, Adleman (RSA)
- Digital Signature Algorithm (DSA)
- Elliptic Curve Cryptosystem (ECC)

A. RSA

- most widely used general **public key** encryption algorithm.
- first published by Rivest, Shamir and Adleman of MIT in 1978
- **Principle** : No mathematical method is yet **known** to efficiently find the prime factors of large numbers
- In RSA, the private and public keys are **constructed** from very **large prime numbers** (consisting of hundred of decimal digits)
 - RSA involves a public-key and a private-key where the public key is known to all and is used to **encrypt data** or message.
 - One of the keys can be made public
 - The data or message which has been **encrypted** using a public key can only be **decrypted** by using its corresponding private-key.

- **RSA**
 - Used for both public key encryption and digital signatures.
 - Security is based on the difficulty of factoring large integers.
- **Major Activities**
 - Key Generation (Algorithm)
 - Encryption
 - Digital signing
 - Decryption
 - Signature verification

- Each user generates a key pair i.e. public and private key using the following steps
 - *each user selects two large primes at random - p, q*
 - *compute their system modulus $n=p.q$*
 - *calculate $\phi(n)$, where $\phi(n)=(p-1)(q-1)$*
 - *selecting at random the encryption key e , where $1 < e < \phi(n)$, and $\gcd(e, \phi(n))=1$*
 - *solve following equation to find decryption key d : $e.d=1 \bmod \phi(n)$ and $0 \leq d \leq n$*
 - *publish their public encryption key: $KU=\{e,n\}$*
 - *keep secret private decryption key: $KR=\{d,n\}$*

- Each entity A creates a public key and a corresponding private key by doing the following.
- Who is capable of **computing public key**?
 - Generate three prime numbers **p** and **q**, **e**
 - Compute $n = p * q$
 - A's Public key: **(e, n)** to be published
- Who is capable of **computing d**?

$$d = \frac{\text{gcd}(\Phi(n)) * \Phi(n) + 1}{e}$$

where

$$\phi(n) = (p-1)(q-1)$$

- To verification d : we must $e * d \bmod \text{theta}(n) = 1$

Example 1: Let, $p=3$, $q=17$, $e=5$: Find the public and Secret key then cipher the message ABC:-

- calculate each keys

$$n = p * q = 3 * 17 = 51$$

Public key ----- $\langle e, n \rangle = \langle 5, 51 \rangle$

$$d = (\text{GCD}(\Phi(n)) * \Phi(n) + 1) / e$$

$$\Phi(n) = (p-1)(q-1) = (3-1)(17-1) = (2*16) = 32$$

$$\text{GCD}(\Phi(n)) = \text{GCD}[(p-1)(q-1)] = \text{GCD}(2, 16) = 2$$

$$d = ((2*32+1)/5) = (64+1)/5 = 65/5 = 13$$

Secret Key ----- $\langle d, n \rangle = \langle 13, 51 \rangle$

To verification d :

$$e \cdot d \bmod \Phi(n) = 1$$

$$5 * 13 \bmod 32$$

$$65 \bmod 32 = 1$$

● Example2

Depending the above example if you have given a message(M)= 7 then encrypt and decrypt the message?

- from the above we get the value for $e=5$ and $d= 13$ and $n=51$
- Now to encrypt the message $m = 7$ is
$$c = m^e \bmod n = 7^5 \bmod 51 = 16807 \bmod 51 = 28$$
Hence the ciphertext $c = 28$
- to decrypt the message $m = 7$
we use $m = C^d \bmod n = 28^{13} \bmod 51 = 38$

- **Example**

- Let given that

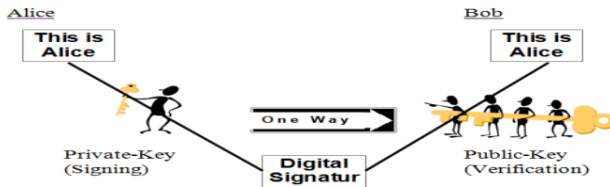
Public key = (n, e) = (17947, 3)

Private key = (n, d) = (17947, 11787)

- encrypt the a message "**ATT**" or that represents (513).
 $c = m^e \bmod n = 513^3 \bmod 17947 = 8363$
- We can verify that our private key is valid by decrypting
 $m = c^d \bmod n = 8363^{11787} \bmod 17947 = 513$

B. Digital Signature Algorithm (DSA)

- **Sender** encrypts message with their **private key**
- **Receiver** can decrypt using **senders** public key
- This authenticates sender, who is only person who has the matching key
- Does not give privacy of data i.e. Decrypt key is public key



8

C.Diffie-Hellman Key Agreement (DH)

- One of the main uses of DH is in the **Internet Key Exchange (IKE) protocol**, a central part of the IP Security(IPSEC) architecture.

Example:

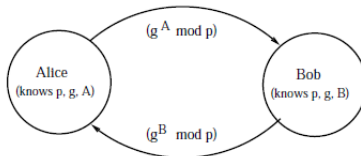
Suppose Alice and Bob want to agree on a shared symmetric key. Alice and Bob, and everyone else, already know the values of **p** and **g**. Alice **generates a random private value** **a** and Bob generates **a random private value** **b**. Both a and b are drawn from the set of integers 1, ..., p-1.

- Alice and Bob derive their corresponding public values,- the values they will send to each other unencrypted-as follows.
 - Alice's public value is $g^a \bmod p$ and Bob's public value is $g^b \bmod p$ then, **they exchange their public values**. Finally,
 - Alice computes $g^{ab} \bmod p = (g^b \bmod p)^a \bmod p$ and Bob computes $g^{ba} \bmod p = (g^a \bmod p)^b \bmod p$.

Chapter 2: Overview of Cryptography

Review of Shared Key Cryptography and Hash Functions

- the first practical method for **establishing** a shared secret over an unsecured communication channel.
- The point is to **agree** on a key that two parties can use for asymmetric encryption, in such a way that an eavesdropper **cannot** obtain the key.



- The steps in this Algorithm
 - 1 Alice and Bob agree on a prime number p and a base g .
 - 2 Alice chooses a secret number a , and sends Bob $(g^a \bmod p)$.
 - 3 Bob chooses a secret number b , and sends Alice $(g^b \bmod p)$.
 - 4 Alice computes $((g^b \bmod p)^a \bmod p)$.
 - 5 Bob computes $((g^a \bmod p)^b \bmod p)$.
- Both Alice and Bob can use this number as their key. Notice that p and g need not be protected.

- **Example**

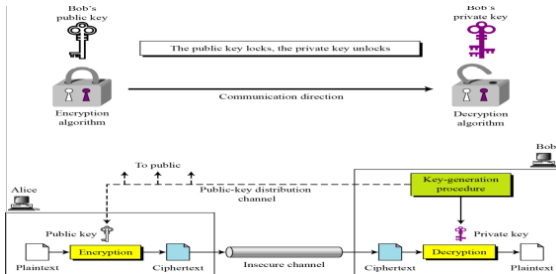
- 1 Alice and Bob agree on $p = 23$ and $g = 5$.
 - 2 Alice chooses $a = 6$ and sends $5^6 \bmod 23 = 8$.
 - 3 Bob chooses $b = 15$ and sends $5^{15} \bmod 23 = 19$.
 - 4 Alice computes $19^6 \bmod 23 = 2$.
 - 5 Bob computes $8^{15} \bmod 23 = 2$.
- Then 2 is the **shared secret**. Clearly, much larger values of **a**, **b**, and **p** are required. An eavesdropper cannot discover this value even if she knows p and g and can obtain each of the messages.

Asymmetric-key Cryptosystem

- In **public key cryptography**, there are two keys: a private key and a public key.
 - public key is announced to the public; whereas the private key is kept by the receiver.
 - The sender **uses** the public key of the receiver for encryption and the receiver uses his private key for decryption
- It is also known as **public-key encryption**.
- **Asymmetric encryption**: can be used for **confidentiality, authentication** or both.

9

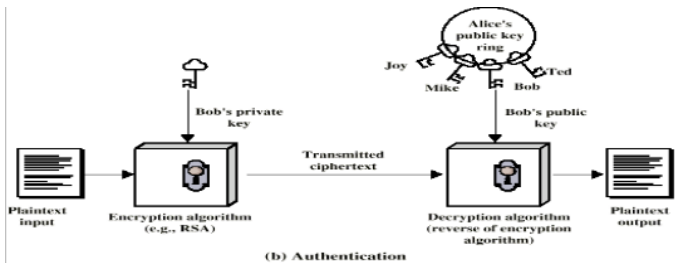
- The fig below shows general idea of **asymmetric-key cryptography** as used for confidentiality.



- The essential steps are the following.

- Each user generates a **pair of keys** to be used for the encryption and decryption of messages.
- Each user places one of the two keys in a **public register** or other accessible file .this is the **public key** .The companion key is **kept private**.
 - If Bob wishes to send a **confidential message** to Alice, Bob encrypts the message using Alice's public key.
 - When Alice receives the message, she **decrypts** it using her **private key**. No other **recipient** can decrypt the message because only Alice knows Alice's private key.

- **Examples:** Public-key Cryptosystems



Key management

- To use symmetric-key cryptography, a **shared secret** key needs to be established between the two parties.
 - To use asymmetric-key cryptography, **each entity** needs to **create** a pair of keys and distribute the public key securely to the community.
- **Key management** defines some **procedures** to create and distribute keys securely.

Key distribution center(KDC)

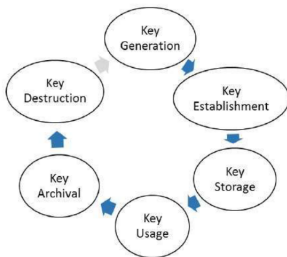
- A practical solution is the use of a **trusted third party**, referred to as a **key-distribution center (KDC)** .
 - Each person establishes a **shared secret key** with the KDC.
 - A **secret key** is established between the KDC and each member. The process is as follows.
 - **Alice** sends a request to the **KDC** stating that she needs a **session(temporary)** secret key between herself and Bob.
 - The KDC **informs** Bob about **Alice's request**.
 - If Bob agrees, a session key is created between the two.
- Note** : - A session symmetric key between two parties is used only once.

Overview of PKI

Public key Infrastructure (PKI)

- The most distinct feature of Public Key Infrastructure PKC is that it uses a pair of keys to achieve the underlying security service.
- The key pair comprises of private key and public key. Since the public keys are in open domain, they are likely to be abused.
- It is, thus, necessary to establish and maintain some kind of trusted infrastructure to manage these keys.
- **Key Management**
 - It goes without saying that the security of any cryptosystem depends upon how securely its keys are managed.
 - Without secure procedures for the handling of cryptographic keys.
 - There are some important aspects of key management which are as follows

- There are some important aspects of **key management** which are as follows
 - Cryptographic keys are nothing but special pieces of data.
 - Key management refers to the secure administration of cryptographic keys.
 - Key management deals with entire key lifecycle as depicted in the following illustration



- There are two **specific requirements of key management** for public key cryptography.
 - **Secrecy of private keys.** Throughout the key lifecycle, secret keys must remain secret from all parties except those who are owner and are authorized to use them.
 - **Assurance of public keys.**
 - In public key cryptography, the public keys are in open domain and seen as public pieces of data. By default there are no assurances of whether a public key is correct, with whom it can be associated, or what it can be used for.
 - Thus key management of public keys needs to focus much more explicitly on assurance of purpose of public keys

● Public Key Infrastructure PKI

- PKI provides assurance of public key.
- It provides the identification of public keys and their distribution.
- An anatomy of PKI comprises of the following components.
 - Public Key Certificate, commonly referred to as 'digital certificate'.
 - Private Key tokens.
 - Certification Authority.
 - Registration Authority.
 - Certificate Management System.

Generally Public Key Infrastructure(PKI)

- a system consisting of set of **hardware** and **software** used for the **management of public key** and **distribution of digital certificates** which are used to **verify** a particular public key **belongs** to a certain entity.
 - **authenticating** users and devices in the digital world.
 - With HTTPS, certificates **serve** to identify the web site you are **connecting** to, to ensure that **no-one** can eavesdrop on your connection.
- **Public key cryptography** : is used to **transmit** user's public key in PKI environment. ¹⁰

- The most **distinct feature** of PKI is that it **uses a pair of keys** to achieve the **underlying security service**.
 - key pair **comprises** of private key and public key. since the public keys are in **open domain** ,it can be **abused** .
- It is ,necessary to establish and maintain some kind of **trusted infrastructure** to manage these keys.
 - PKI provides **assurance** of public key.
 - PKI provides the **identification** of public keys and their distribution.
- **Public key cryptography** : is used to **transmit** user's public key in PKI environment.

Public-key distribution

- In Asymmetric-key cryptography, people **do not** need a symmetric shared key.

Example

- If Alice wants to send a message to Bob, she **only needs** to know Bob's public key, which is **open** to the public and available to everyone.
- If Bob needs to send a message to Alice, he **only needs** to know Alice's public key, which is also known to everyone.
 - In **public-key cryptography** , everyone **shields** a private key and **advertises** a public key.
- **Note:**In public-key cryptography, everyone has **access** to everyone's public key – public keys are available to the public.

Public announcement

- The naive approach is to **announce public keys publicly**.
- **Example:**
Bob can put his public key on **his web site** or announce it in a local or national newspaper. When **Alice** needs to send a **confidential message** to Bob, she can obtain Bob's public key from his site or from the newspaper, or even send a message to ask for it. This approach, however, is not secure:it is **subject to forgery**.

Trusted center

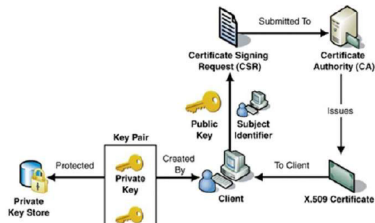
- A more secure approach is to have a **trusted center** retain a directory of public keys.
- The directory, like the one used in a telephone system, is dynamically updated.
 - Each user can **select** a private and public key, keep the private key, and deliver the public key for **insertion** into the directory.

- A PKI consists-
 - Public Key Certificate, commonly referred to as **digital certificate**
 - **Certification Authorities(CAs)**
 - Registration **Authorities (RAs)**
 - Certificate Management System
 - Private Key tokens
- **A Public Key Infrastructure ensures the following:**
 - Ensures the **quality of information** transmitted over the network.
 - Lifetime and validity of the information.
 - Certainty of the privacy, and source and destination of that information.
 - To ensure non repudiation.

A. Digital Certificate(DC)

- For analogy, a certificate can be considered as the ID card **issued** to the person i.e ID cards such as a driver's license, passport to prove their identity.
- DC does the same basic thing in the electronic world, but with one difference.
 - DC are **not only** issued to people but they can be issued to computers, software packages or anything else that need to prove the identity in the electronic world.
 - DC are based on the **ITU standard X.509** which defines a **standard certificate format** for public key certificates and certification validation.
- Public key pertaining to the user client is stored in digital certificates by the **Certification Authority(CA)** along with other relevant information such as client information, expiration date, usage, issuer etc.

- CA digitally **signs** this entire information and includes digital signature in the certificate.
- Anyone who **needs** the assurance about the public key and associated information of client, he carries out the signature validation process using CA's public key.
- The process of obtaining Digital Certificate by a person/entity is depicted in the following illustration.



B. Certification Authorities(CA)

- CA is an entity claimed(by someone) to be trustworthy for verifying identities and issuing public key certificates.
- There are **commercial CAs**, **governmental CAs**, and even **free CAs**.
 - To **use** a CA, you must know its **own key**. You can learn that CA's key, if you can obtain a chain of **CA-signed certificates** that starts with a CA whose key you already know.
- The previous approach can create a **heavy load** on the center **if the number of requests is large**. **soln**:public key certificate

- Look the following clear example

Example

- Bob wants two things: he wants people to know his **public key**, and he wants no-one to accept a **forged public key** as his.
 - Bob can go to a **certification authority(CA)**, a government authority that binds a public key to an entity and issues a certificate.
- The CA itself **has** a well known public key that **cannot** be forged. The CA issues a certificate for Bob.
- To prevent the certificate itself from **being forged**, the **CA signs** the certificate with its **private key**. Now Bob can upload the signed certificate.
 - Anyone who wants Bob's **public key downloads** the signed certificate and uses the center's public key to extract Bob's public key.

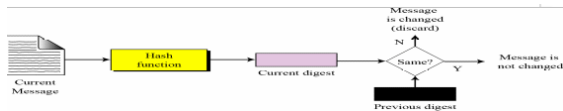
Message integrity

- One way to **preserve** the integrity of a document was traditionally through the use of a **fingerprint**.
 - electronic equivalent of the document and fingerprint pair is the **message** and **digests pair**.
 - to preserve the integrity of a message, the message is passed through an algorithm called a **cryptographic hash function**.
- a function **creates** a compressed image of the message that can be used like a fingerprint.



Checking integrity

- To **check** the integrity of a message or document, we **run** the cryptographic hash function again and compare the new message digest with the previous one.
 - If both are the **same**, we are sure that the original message has **not** been changed.



Message authentication

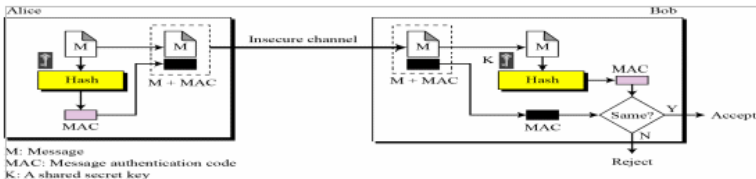
- **A message digest** guarantees the integrity of a message/it guarantees that the message has not been changed.
- A message digest, however, **does not authenticate** the sender of the message.

When Alice **sends** a message to Bob, Bob needs to know that the message is really from Alice. To **provide message authentication**, Alice needs **to provide proof** that it is she who is sending the message and not an **impostor**.

- A message digest peruse **cannot** provide such a proof. the digest created by a **cryptographic hash function** is normally called a **modification detection code(MDC)**.
- What we need for message authentication is a **message authentication code(MAC)**.

Message authentication code(MAC)

- To ensure the integrity of the message and authenticate its origin, we need to change an MDC to a MAC.
- The difference is that the latter includes a secret between Alice and Bob.



Introduction to Hash Function

- transformation of a **string of characters** into a usually **shorter fixed-length value** or key that represents the original string.
- A hash value (or simply hash), also called a **message digest**, is a number generated from a string of text. The hash is substantially smaller than the text itself.
- In hashing, a **fixed-length message digest** is created out of a **variable-length message**.
 - also refers to a **search technique** or a method of **accessing data** records, where search time is independent of the number of the elements in the collection.
 - creates a **unique, fixed-length signature** for a message or data set.

- **Hashing:** is used to **index** and **retrieve items** in a database because it is faster to find the item **using** the shorter hashed key than to find it using the original value.
 - **Example:**
consider a list of **names in a database**: A database search mechanism would first have to start looking **character-by-character** across the name for matches until it found the match. But if each of the **names were hashed**, it might be possible to generate a unique four-digit key or index for each name. So you might get something like: 1345 Russel Akram

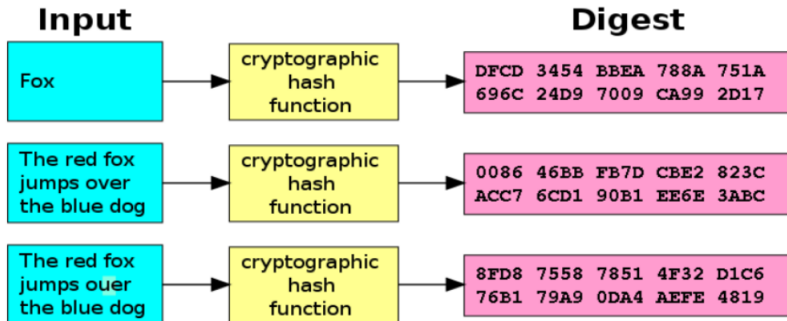
Importance of Hashing

- Hashing is used to **index** and **retrieve** items in a database because it is faster to find the item using the shorter hashed key than to find it using the original value.
- Hashing is also used to encrypt and decrypt digital signatures (used to authenticate message senders and receivers).
- it is used to **ensure** that transmitted messages have not been tampered with.

Cryptographic hash function

- **Cryptographic hash function**: creates hash value in such a way that any (accidental or intentional) **change to the data will change the hash value**.
- data to be encoded are often called the **message**, and the hash value is sometimes called the **message digest/digest**.
 - In cryptographic hash function, even a **small changes** in the input would **cause** a large change in the output.

- Figure below shows how the slight changes input (here in the word "over") drastically change the resulting output.



The two commonly used hash functions are MD5 and SHA-1.

- **MD5 (Message Digest):**
 - Several MD hash algorithms designed by Ron Rivest are MD2, MD4 and MD5.
 - The last version MD5 is more secured than the previous versions.
 - It divides the message into blocks of 512 bits and creates a 128-bit digest.
- **SHA-1 (Secure Hash Algorithm):**
 - This standard was developed by NIST (National Institute of Standards and Technology).
 - This standard is mostly based on MD5.
 - Several versions of SHA standard were realised: SHA-1, SHA-224, SHA-256, SHA-384 and SHA-512.
 - SHA-1 returns a string of 160 bits.

Application of Hash Function in Cryptography

- **Verifying the integrity of message and file:**
 - **Determining whether any changes have been made to a message**
- **Verifying password for secure login:**
 - When the password is created , the **system hashes** it and **stores** the hash in the password file.
 - When the user sends user ID and password, the **system creates a hash** of the password and then compare the hash value with the one stored in the file.
- **File or data identifier:**
 - reliably identifying a file
 - to allow the fast look-up of a data in a hash table
- **authentication:** assurance that the communicating entity is the one that it claims to be.
- **digital signatures**

Some Popular Hash Function

- Here are some relatively simple hash functions that have been used:
Division-remainder method, Mid-square method and Folding method

A. Division-remainder method:

- Using this method, choose a number m that is larger than the number n of keys in K (K is a set of keys).
- Generally, the number m is chosen to be a prime number.
The hash function H is defined as:
 $H(k) = k \pmod{m}$ or, $H(k) = k \pmod{m} + 1$
- Here $k \pmod{m}$ denotes the **remainder** when k is divided by m .
- The second formula is used when we want the **hash addresses** to range from **1 to m** rather than from **0 to $m-1$** .

- **Example: Division-remainder method:**

Suppose a company with **68** employees assigned a **4-digit** employee number to each employee which is used as the primary key. Apply the **division method of hash function** to each of the following employee number: 3205, 7148, 2345.

Solution:

Since, there are 68 employees in the company, two digit employee number is sufficient to represent them.

Highest 2 digit number is 99 and 97 is the nearest 2 digit prime number of 99. So, we divide each of the 4 digit employee number by 97.

$$H(3205) = 3205 \pmod{97} = 04$$

$$H(7148) = 7148 \pmod{97} = 67$$

$$H(2345) = 2345 \pmod{97} = 17$$

- In the case that the **memory addresses begin with 01** rather than 00, we choose that the function $H(k) = k(modm) + 1$ to obtain:

$$H(3205) = 3205(mod97) + 1 = 4 + 1 = 05$$

B. Mid-square method:

- Using this method, the key k is squared. Then the hash function H is defined by: $H(k) = I$, Where I is obtained by **deleting digits** from both ends of k^2 . from the example of the above question:

K	3205	7148	2345
K^2	102 72 025	510 93 904	54 99 025
$H(k) = I$	72	93	99

C.Folding method

- The key **k** is portioned into a number of parts, $k_1, k_2, k_3, \dots, k_r$.
 $H(k) = k_1 + k_2 + k_3, \dots + k_r$, where the **leading-digit carries**, if any, are ignored.

From the example of the above question:

Solution:

K	3205	7148	2345
$H(k)=K_1+k_2$	$32+05=37$	$71+48=119$	$23+45=77$
H(k)	37	19	77

- Alternatively, reverse folding method may be used as:
 $H(3205) = 32 + 50 = 82$

Thank You !!!

